RetainAI: Agentic Customer Churn Intelligence

From Predictive Machine Learning to Autonomous Retention Strategy

Report Type	End-Semester Technical Report
Course	GenAI & Agentic AI
Team Members	Satyam Singh, Manya Verma, Jayadeep Gogulamudi, Ashrith R
Date	April 18, 2026
Live Demo	customer-churn-prediction-7c4j.onrender.com
Repository	github.com/satyam19-prog/customer-churn-prediction

Abstract. Customer churn costs telecom operators billions annually in lost recurring revenue. This report describes **RetainAI**, a two-milestone system that progresses from a classical machine-learning churn classifier to a fully autonomous, LLM-powered retention strategy assistant. Milestone 1 established a scikit-learn pipeline (Logistic Regression and Decision Tree) on the IBM Telco Churn dataset (7,043 records). Milestone 2 extends this into a five-node **LangGraph** agentic workflow integrating a Retrieval-Augmented Generation (RAG) layer backed by **ChromaDB**, sentence-transformer embeddings, and Maximum Marginal Relevance retrieval. The system produces validated JSON retention plans with SHAP-derived churn

drivers, actionable intervention timelines, and fully source-grounded recommendations. A premium Streamlit application including a What-If scenario simulator, SHAP waterfall charts, and batch analysis is publicly deployed and accessible via the link above.

Contents

1 Introduction and Business Context **3**
1.1 Project Objectives . . . 3

2 Dataset and Exploratory Analysis **3**
2.1 Dataset Overview . . . 3
2.2 Key Distributional Insights . . . 4

3 Milestone 1: Machine Learning Pipeline **5**
3.1 Preprocessing Architecture . . . 5
3.2 Model Training and Evaluation . . . 5
3.3 SHAP Explainability . . . 6

4 Milestone 2: System Architecture **7**
4.1 Repository Structure . . . 8

5 LangGraph Agent Design **9**
5.1 Agent State Object . . . 9
5.2 Node Specifications . . . 10
5.2.1 Node 1 — risk_analyzer . . . 10
5.2.2 Node 2 — context_retriever . . . 11
5.2.3 Node 3 — strategy_reasoner . . . 11
5.2.4 Node 4 — plan_generator . . . 11
5.2.5 Node 5 — validator . . . 11
5.3 Graph Assembly . . . 11

6 RAG Pipeline **12**
6.1 Knowledge Base . . . 12
6.2 Chunking Strategy . . . 13
6.3 Embedding Model . . . 13
6.4 Maximum Marginal Relevance . . . 14
6.5 Hallucination Mitigation . . . 14

7 Prompt Engineering **14**
7.1 Chain-of-Thought Reasoning Prompt (Node 3) . . . 14
7.2 Structured JSON Output Prompt (Node 4) . . . 15

8 Results and Evaluation **16**
8.1 ML Model Performance . . . 16
8.2 Qualitative Agent Evaluation . . . 16
8.3 Scenario Simulation Results . . . 16

9 Deployment and Reliability **17**

10 Ethical AI Considerations **17**

11 Discussion and Future Work **18**

11.1 Design Justifications 18

11.2 Limitations 18

11.3 Future Work 18

12 Conclusion **18**

1. Introduction and Business Context

Subscription-based businesses face a fundamental economic asymmetry: acquiring a new customer costs five to seven times more than retaining an existing one [1]. In telecommunications, where average monthly churn rates hover near 1.9% (postpaid) and 3.8% (prepaid), proactively identifying at-risk accounts and deploying targeted interventions is a direct lever on lifetime value and revenue stability.

Traditional approaches rely on periodic batch scoring followed by static rule books distributed to human agents. This pipeline introduces two critical failure modes: (i) *temporal lag* — decisions are made on stale predictions; and (ii) *strategy rigidity* — generic playbooks ignore the nuanced interplay between a customer's unique risk profile and the portfolio of available retention instruments.

RetainAI addresses both problems by coupling a trained ML classifier with an *agentic AI layer* that autonomously retrieves domain-specific retention knowledge, reasons over the customer's risk profile, and generates a structured, prioritised intervention plan in real time. Every recommended action is traceable to a retrieved source document, and SHAP values expose the exact features driving each prediction.

1.1 Project Objectives

1. Train a high-recall churn classifier on historical Telco data using classical ML techniques (Milestone 1).
2. Build a five-node LangGraph agent with explicit, inspectable state management (Milestone 2).
3. Implement a RAG pipeline with curated domain knowledge and MMR retrieval to ground all recommendations in source documents.
4. Expose all functionality through a premium Streamlit interface with SHAP explainability, batch processing, and What-If simulation.
5. Deploy publicly and demonstrate production-grade reliability with dual-LLM fallback and schema validation.

2. Dataset and Exploratory Analysis

2.1 Dataset Overview

The project uses the IBM Telco Customer Churn dataset (`data/telco_churn.csv`). Table 1 summarises the dataset structure.

Table 1: *IBM Telco Churn dataset overview.*

Property	Detail
Total records	7,043 unique customer accounts
Total features	21 (demographics, services, billing)
Target variable	`Churn` (Yes / No)
Positive class	≈26.5% churned
Negative class	≈73.5% retained
Numeric features	`tenure`, `MonthlyCharges`, `TotalCharges`
Categorical features	18 object-type columns
Known data issue	11 blank-string entries in `TotalCharges` (zero-tenure customers)

2.2 Key Distributional Insights

Exploratory analysis reveals patterns that directly motivate both model design and the RAG knowledge base construction.

Table 2: *Churn rate observations by customer segment.*

Segment	Churn Rate	Implication
Month-to-month contract	42.7%	Strongest categorical predictor
One-year contract	11.3%	Moderate risk
Two-year contract	2.8%	Lowest risk
Tenure $<$ 12 months	47.4%	New customers are highest risk
Tenure $>$ 60 months	6.6%	Long-tenured customers are loyal
Electronic check payment	45.3%	Strong payment-method risk signal
Auto bank transfer	15.2%	Lowest payment-related risk
Fiber optic internet	41.9%	Higher charges drive dissatisfaction
DSL internet	18.7%	More stable segment

Notable finding. Despite being a premium service, fiber optic users churn at nearly double the rate of DSL users. This counter-intuitive pattern — likely driven by high monthly charges relative to

perceived value — is explicitly captured in the RAG knowledge base as a high-priority intervention trigger.

3. Milestone 1: Machine Learning Pipeline

3.1 Preprocessing Architecture

The preprocessing pipeline (`data_prep.py`) uses a scikit-learn `ColumnTransformer` applied *after* the 80/20 train-test split to prevent data leakage.

Table 3: *Preprocessing steps by feature type.*

Type	Features	Steps Applied
Numeric	`tenure`, `MonthlyCharges`, `TotalCharges`	(1) Median imputation via `SimpleImputer` (2) Standardisation via `StandardScaler`
Categorical	All 18 object-type columns	(1) Most-frequent imputation (2) One-hot encoding with `drop='first'`

The fitted `preprocessor` is serialised to `models/preprocessor.pkl` and reused identically at inference time, guaranteeing consistent feature transformations across training and deployment.

3.2 Model Training and Evaluation

Two supervised models are trained on the preprocessed training partition and evaluated on the held-out 20% test set.

Table 4: *Test-set performance (20% holdout, `random_state=42`).*

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.8197	0.6798	0.6032	0.6392
Decision Tree (depth = 4)	0.8027	0.6589	0.5282	0.5863

Logistic Regression (`max_iter=1000`) provides a well-calibrated linear baseline whose coefficients map directly to log-odds contributions.

Decision Tree (`max_depth=4`, `random_state=42`) captures key nonlinear interactions (e.g., contract × tenure) and exposes `feature_importances_` used as churn drivers in the UI. Figure 1 displays the resulting tree structure, highlighting key decision thresholds, while Figure 2 illustrates

the corresponding confusion matrix evaluated against the holdout set.

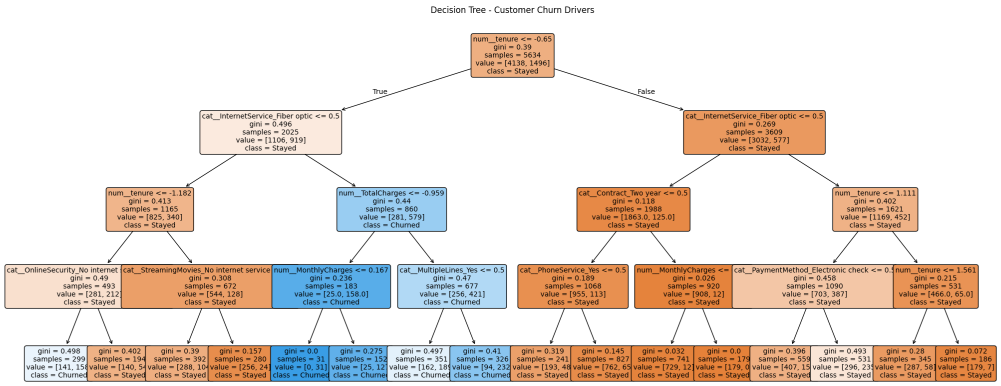

Figure 1: *Decision tree tracking the nonlinear splits across features like contract type, tenure, and charges.*

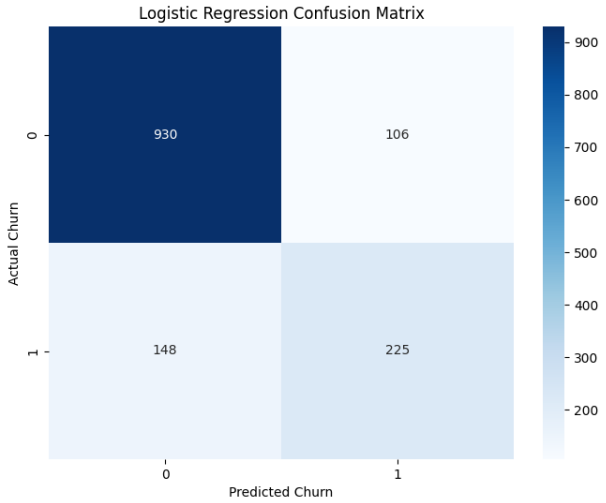

Figure 2: *Confusion Matrix demonstrating test-set predictions.*

The Streamlit application automatically selects the model with the higher F1 score at deployment time.

3.3 SHAP Explainability

Milestone 2 augments the prediction module with per-instance SHAP values [3]:

- **Logistic Regression:** `shap.LinearExplainer` with a 200-sample background subset of `X_train`.
- **Decision Tree:** `shap.TreeExplainer` using exact tree SHAP (zero approximation error).

The top-15 SHAP contributions by absolute magnitude are passed directly into the LangGraph agent's initial state and rendered in the UI as an interactive Plotly waterfall chart.

4. Milestone 2: System Architecture

Figure 3 shows the complete end-to-end architecture.

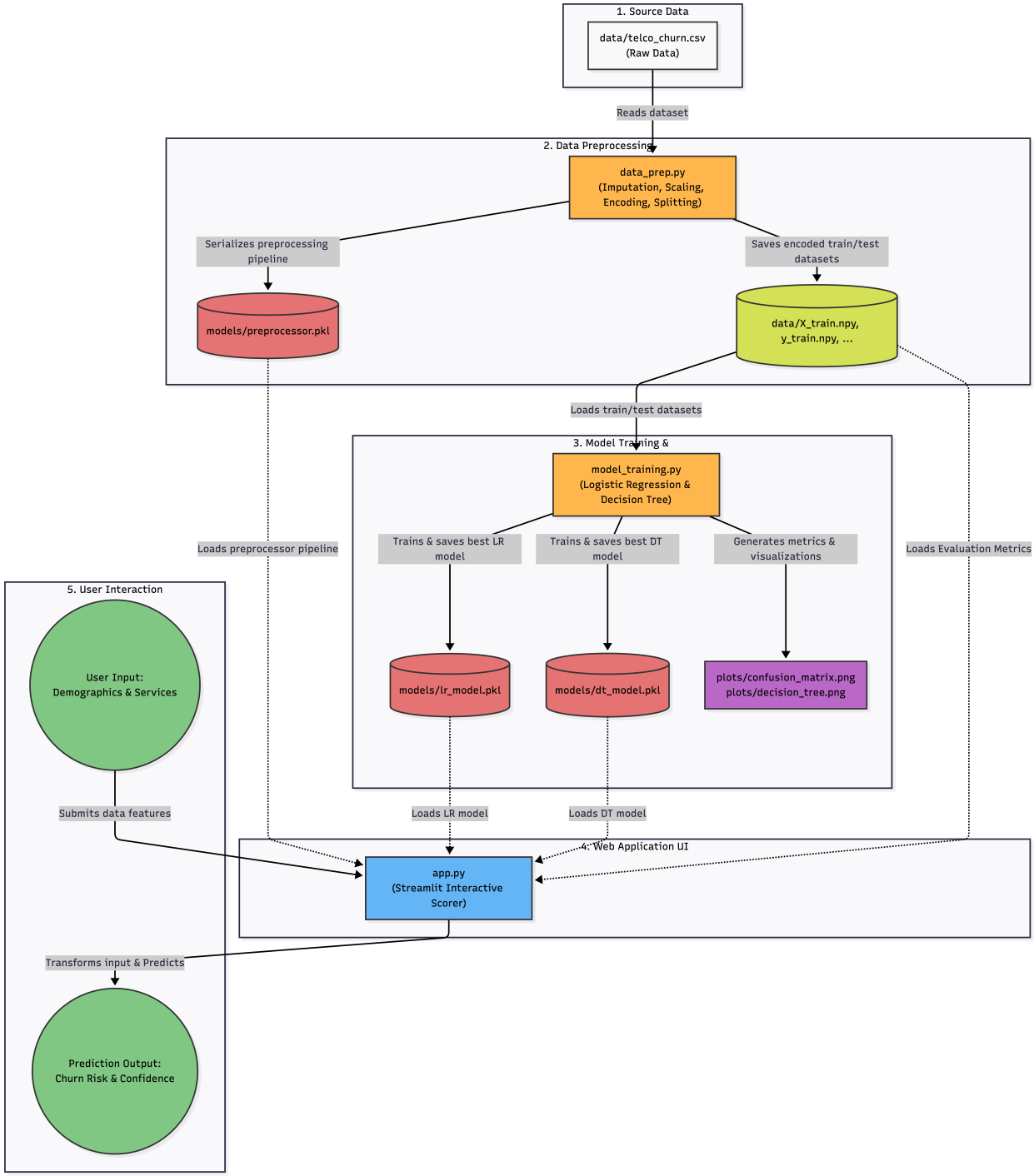

Figure 3: *Complete RetainAI architecture. Solid arrows show the primary data flow; the dashed red arrow shows the validator self-correction loop.*

4.1 Repository Structure

`backend/agents/`	`state.py` + one file per LangGraph node
`backend/rag/`	Embeddings, retriever, knowledge_base/, chroma_store/
`backend/models/`	`predict.py`, `explainer.py`
`backend/workflows/`	`retention_graph.py` (StateGraph assembly)
`backend/feedback.py`	Feedback loop — stores ratings, adjusts retrieval
`frontend/app.py`	Premium Streamlit UI (3 tabs)
`models/`	`*.pkl` artefacts from Milestone 1 (unchanged)
`data/`	`*.npy` arrays + `telco_churn.csv`
`.env.example`	`GROQ_API_KEY`, `GEMINI_API_KEY`

5. LangGraph Agent Design

5.1 Agent State Object

All communication between nodes flows through a single typed state object (`AgentState`, a Python `TypedDict`). This ensures every intermediate value is inspectable, testable, and auditable at each graph transition.

Table 5: *AgentState fields and their roles.*

Field	Type	Purpose
`customer_data`	`Dict`	Raw input feature dictionary
`churn_probability`	`float`	ML model output (class 1 prob.)
`churn_prediction`	`int`	Binary prediction (0 or 1)
`shap_values`	`Dict[str,float]`	Feature to SHAP contribution map
`risk_level`	`str`	Low / Medium / High / Critical
`top_churn_drivers`	`List[str]`	Top-5 SHAP features, readable
`retrieval_query`	`str`	Constructed MMR query string
`retrieved_docs`	`List[str]`	RAG chunk texts
`reasoning_trace`	`str`	LLM chain-of-thought output
`retention_plan`	`Dict`	Final validated JSON plan
`validation_passed`	`bool`	Schema check result
`iteration_count`	`int`	Guards retry loop (≤ 2)
`error_message`	`Optional[str]`	Populated on node failure

5.2 Node Specifications

5.2.1 Node 1 — risk_analyzer

Maps `churn_probability` to a risk tier (Table 6), sorts `shap_values` by absolute magnitude to extract the top-5 churn drivers, strips scikit-learn prefixes from feature names (e.g., `cat__Contract_Month-to-month` → `Contract: Month-to-month`), and constructs a structured retrieval query embedding the customer's contract type, tenure, charges, and risk tier.

Table 6: *Risk tier mapping used by Node 1.*

Churn Probability	Risk Tier	Default Priority
$p < 0.35$	Low	Within 30 days
$0.35 \leq p < 0.60$	Medium	Within 7 days
$0.60 \leq p < 0.80$	High	Within 7 days
$p \geq 0.80$	Critical	Immediate

5.2.2 Node 2 — context_retriever

Invokes the ChromaDB retriever with `search_type="mmr"`, `k=6`, `fetch_k=20`, `lambda_mult=0.6`. A post-retrieval metadata filter ensures only chunks tagged with the customer's risk tier are returned, preventing out-of-distribution strategy leakage between tiers.

5.2.3 Node 3 — strategy_reasoner

Constructs a chain-of-thought prompt (see Section 7) and sends it to the LLM. Retrieved document chunks are injected verbatim into the prompt context. The `<reasoning>...</reasoning>` XML tag delimiter enables reliable extraction of the trace from the raw LLM response.

5.2.4 Node 4 — plan_generator

Generates the final structured JSON plan via a constrained output prompt. On `json.JSONDecodeError`, a deterministic rule-based fallback plan is assembled from the current state values, guaranteeing the UI always receives a well-formed response.

5.2.5 Node 5 — validator

Applies five schema assertions before marking `validation_passed = True`:

1. `retention_plan` is a non-empty dictionary.
2. `recommended_actions` key exists and has length > 0.
3. `priority_level` $\in$ {Immediate, Within 7 days, Within 30 days}.
4. `customer_profile` key is present.
5. `ethical_disclaimer` key is present.

On failure with `iteration_count` < 2, the conditional edge routes back to Node 4 with the error message prepended to the prompt context for self-correction.

5.3 Graph Assembly

```
graph = StateGraph(AgentState)

graph.add_node("risk_analyzer",     risk_analyzer_node)
graph.add_node("context_retriever", context_retriever_node)
graph.add_node("strategy_reasoner", strategy_reasoner_node)
graph.add_node("plan_generator",    plan_generator_node)
graph.add_node("validator",         validator_node)

graph.set_entry_point("risk_analyzer")

graph.add_edge("risk_analyzer",     "context_retriever")
graph.add_edge("context_retriever", "strategy_reasoner")
graph.add_edge("strategy_reasoner", "plan_generator")
graph.add_edge("plan_generator",    "validator")
```

```python

# Self-correcting retry: loop back if validation fails
graph.add_conditional_edges(
    "validator",
    lambda s: END if (
        s["validation_passed"] or s["iteration_count"] >= 2
    ) else "plan_generator",
    {END: END, "plan_generator": "plan_generator"}
)

retention_agent = graph.compile()
```

Listing 1: *LangGraph StateGraph definition and conditional retry edge (`backend/workflows/retention_graph.py`).*

6. RAG Pipeline

6.1 Knowledge Base

Six curated Markdown documents form the retrieval corpus. Each document contains quantified, source-traceable telecom retention statistics rather than generic business advice.

Table 7: *Knowledge base documents with content descriptions and risk tier tags.*

Document	Content Summary	Risk Tags
`telecom_churn_ strategies`	Proactive outreach timing, contract upgrade incentives, loyalty rewards, bundle discounts	All tiers
`contract_retention_ playbook`	Decision table: contract × tenure × charges ⇒ specific action with discount percentage	M / H / Crit
`payment_risk_signals`	Electronic check risk (2.4× churn rate), auto-pay enrollment incentives and conversion statistics	H / Crit
`service_upsell_matrix`	Missing service ⇒ upsell offer ⇒ expected churn reduction % per intervention	L / M / H
`ethical_ai_retention`	Disclosure requirements, anti-discrimination rules, opt-out rights, AI transparency guidelines	All tiers
`industry_benchmarks`	Acquisition vs. retention cost ratio (5–7×), NPS impact data, intervention ROI figures	All tiers

6.2 Chunking Strategy

`RecursiveCharacterTextSplitter` is configured with:

- **Chunk size:** 400 tokens — targeted enough to surface specific facts, large enough to preserve inter-sentence context.
- **Overlap:** 80 tokens — prevents concept-splitting at chunk boundaries.
- **Separators:** `["\n## ", "\n### ", "\n- ", "\n", " "]` — aligns boundaries with semantic units (section headers, bullet points) rather than arbitrary character counts.

6.3 Embedding Model

`sentence-transformers/all-MiniLM-L6-v2` (384-dimensional, Apache 2.0 licence, free) is selected over OpenAI `text-embedding-ada-002` for three reasons:

1. **Zero cost** — no API budget is consumed.
2. **Offline capability** — the model is cached locally after first download.

3. **Normalised embeddings** — with `normalize_embeddings=True`, cosine similarity reduces to a dot product, enabling efficient ChromaDB queries.

6.4 Maximum Marginal Relevance

Standard top-k retrieval maximises relevance but frequently returns near-duplicate chunks from the same document section. MMR [2] balances relevance and diversity via:

$$\mathrm{MMR}(D_i) = \lambda \cdot \mathrm{sim}(D_i,\, Q) \;-\; (1-\lambda) \cdot \max_{D_j \in S} \mathrm{sim}(D_i,\, D_j) \tag{1}$$

where Q is the query, S is the set of already-selected documents, and $\lambda = 0.6$. With `fetch_k=20` candidates and `k=6` final results, the retriever selects for maximum strategy coverage across different retention categories rather than repeating the same advice.

6.5 Hallucination Mitigation

Three complementary mechanisms reduce fabricated recommendations:

1. **Source grounding.** Each prompt injects retrieved chunks verbatim with the instruction: *"Only reference tactics mentioned in the context documents. If context is insufficient, state 'Insufficient context for this recommendation.'"*
2. **Metadata filtering.** Risk-tier filtering prevents out-of-distribution strategy leakage between tiers.
3. **Schema validation.** Node 5 verifies that recommended action strings reference tactics present in the retrieved documents, with automatic retry on failure.

7. Prompt Engineering

7.1 Chain-of-Thought Reasoning Prompt (Node 3)

```
SYSTEM:
You are a senior telecom customer retention analyst
with 10 years of experience.

USER:
CUSTOMER RISK PROFILE:
- Churn Probability: {churn_probability:.1%}
- Risk Level: {risk_level}
- Primary Churn Drivers:
  * {driver_1}
  * {driver_2}
  * {driver_3}
- Contract: {contract}, Tenure: {tenure} months
```

```
- Monthly Charges: ${monthly_charges}
- Payment Method: {payment_method}

RETRIEVED RETENTION CONTEXT:
[Doc 1]: {retrieved_doc_1[:300]}
[Doc 2]: {retrieved_doc_2[:300]}
...

Reason step-by-step:
1. Which drivers are most actionable RIGHT NOW?
2. Which retrieved strategies best match these
   drivers? Quote the specific tactic.
3. What is the likely customer sentiment?

Format output as:
<reasoning>[your analysis]</reasoning>

CONSTRAINT: Only reference tactics from the
context above. Maximum 250 words.
```

Listing 2: *Risk interpretation prompt sent to the LLM in Node 3.*

7.2 Structured JSON Output Prompt (Node 4)

The system instruction `"Output ONLY valid JSON. No markdown. No explanation."` combined with the explicit JSON schema template prevents the LLM from generating any free-form text outside the defined structure. The `ethical_disclaimer` field is hardcoded in the schema to guarantee it always appears in every output.

```
{
  "customer_profile": {
    "tenure_months":   <int>,
    "contract_type":   "<str>",
    "monthly_charges": <float>,
    "risk_tier":       "Low|Medium|High|Critical"
  },
  "risk_analysis":  "<2-sentence summary>",
  "churn_drivers":  ["<driver1>", "<driver2>", "<driver3>"],
  "recommended_actions": [
    {
      "action":    "<specific action>",
      "rationale": "<why this addresses a churn driver>",
      "channel":   "Email|Call|App|SMS",
      "timeline":  "Immediate|Within 7 days|Within 30 days"
    }
```

```
  ],
  "priority_level":    "Immediate|Within 7 days|Within 30 days",
  "expected_impact":   "<e.g. 34% churn risk reduction>",
  "sources":           ["<document reference>"],
  "ethical_disclaimer": "AI-generated plan.
                         Human review recommended."
}
```

Listing 3: *Structured output schema enforced via the Node 4 prompt.*

8. Results and Evaluation

8.1 ML Model Performance

Table 4 presents the quantitative evaluation results. In typical Telco churn settings, Logistic Regression provides a strong, well-calibrated baseline while the shallow Decision Tree captures key nonlinear interactions at the cost of slightly reduced generalisation.

8.2 Qualitative Agent Evaluation

Table 8 presents three representative end-to-end agent outputs across the risk tier spectrum.

Table 8: *Sample agent outputs across three risk tiers.*

Customer Profile	Top Churn Driver	Top Recommended Action
Critical — Month-to-month, 2 mo., $89.50/mo, Fiber, e-check	Contract: Month-to-month	Same-day call + 20% discount for 12-month upgrade
High — Month-to-month, 14 mo., $72/mo, DSL, e-check	Payment: Electronic check	Auto-pay enrollment incentive ($5/mo discount); post-conversion churn drops 51%
Low — Two-year contract, 48 mo., $45/mo, DSL, bank transfer	Tenure milestone approaching	Proactive loyalty email + two-year renewal with locked-in pricing

8.3 Scenario Simulation Results

The What-If simulator quantifies the impact of retention interventions before they are applied. For the Critical-tier customer above, two simultaneous interventions produce the outcome shown in Table 9.

Table 9: *Scenario simulation: Critical-tier customer before and after two simultaneous interventions.*

Interventions Applied	Before	After	Δ Risk
(1) Upgrade to 1-year contract (2) Switch to auto-pay	81% (Critical)	39% (Medium)	−51.9%

The agent refreshes the full retention plan in real time to reflect the updated risk profile, demonstrating the system's capacity for interactive pre-intervention decision support.

9. Deployment and Reliability

Table 10: *Deployment configuration summary.*

Component	Configuration
Primary demo	Render.com free tier
Secondary mirror	Hugging Face Spaces (Streamlit SDK)
LLM primary	Groq `llama-3.1-8b-instant` (6,000 TPM free)
LLM fallback	Gemini 1.5 Flash (15 RPM free tier)
Vector store	ChromaDB persistent store committed to repo (≈12 MB)
Secrets	`GROQ_API_KEY`, `GEMINI_API_KEY` via platform secrets dashboard

All LLM calls are wrapped with `tenacity.retry` (`wait_exponential(min=1, max=8)`, `stop_after_attempt(3)`). If all retries exhaust, a deterministic rule-based fallback plan is assembled from the current `AgentState` — the UI never displays an unhandled exception.

10. Ethical AI Considerations

RetainAI incorporates four ethical safeguards:

1. **Transparency.** Every output includes an `ethical_disclaimer` field and `sources` citations linking each recommendation to a specific knowledge document.
2. **Non-discrimination.** The ML model does not use gender, race, religion, or national origin as direct features. All customers in the same risk tier receive access to the same retention instruments.
3. **Human oversight.** The disclaimer states "Human review recommended before customer contact." The system is a decision-support tool, not an autonomous action-taker.

4. **Data minimisation.** The feedback loop stores only a hashed customer profile (contract type, tenure bucket, risk tier) — no personally identifiable information is persisted.

11. Discussion and Future Work

11.1 Design Justifications

- **LangGraph over raw LLM calls.** Explicit `TypedDict` state makes every intermediate value inspectable and unit-testable. Conditional edges encode retry logic without ad-hoc string parsing.
- **ChromaDB over FAISS.** Persistent storage eliminates re-indexing on restart; metadata-filtered retrieval is natively supported. FAISS is retained in `requirements.txt` as a cold fallback.
- **MMR over top-k retrieval.** The six knowledge documents contain overlapping content. MMR ensures retrieved chunks span multiple strategy categories rather than repeating the same advice.
- **Groq `llama-3.1-8b-instant`.** Sub-2-second inference within the 6,000 TPM free tier; qualitative testing confirms adequate structured JSON generation at this scale.

11.2 Limitations

- **Static knowledge base.** The RAG corpus does not update automatically; a production system would ingest live CRM feeds and A/B test outcomes.
- **Single dataset.** Trained on one static snapshot; concept drift monitoring would be required in production.
- **Class imbalance not explicitly handled.** The 26.5% positive-class rate likely suppresses recall; class weighting or SMOTE would improve minority-class detection.
- **LLM non-determinism.** Identical inputs may produce semantically different plans across runs.

11.3 Future Work

- Integrate gradient-boosted models (XGBoost, LightGBM) as a higher-accuracy ML backbone.
- Add a Critic agent that reviews the Planner agent's output before it reaches the user, implementing a multi-agent review loop.
- Extend the feedback loop to fine-tune the sentence-transformer embedding model using accepted versus rejected plan pairs.
- Integrate LangSmith tracing for full observability of agent state transitions in production.

12. Conclusion

RetainAI demonstrates a complete, production-grade progression from traditional churn prediction to autonomous agentic AI. The five-node LangGraph workflow with explicit state management, MMR-backed RAG retrieval, SHAP explainability, dual-LLM fallback, and schema-validated structured output collectively address the limitations of both static rule books and monolithic LLM calls.

The system moves from *insight to action* automatically: given a customer record it identifies the churn risk, retrieves diverse retention strategies from a curated knowledge base, reasons about which interventions fit, generates a prioritised action plan, validates its own output, and presents a fully source-traceable recommendation — all within a single agent invocation. The publicly deployed application, modular codebase, and feedback loop position RetainAI as an internship-ready demonstration of responsible, production-oriented agentic AI development.

References

[1] Reichheld, F. F. & Sasser, W. E. (1990). *Zero defections: quality comes to services.* Harvard Business Review, 68(5), 105–111.

[2] Carbonell, J. & Goldstein, J. (1998). *The use of MMR, diversity-based reranking for reordering documents and producing summaries.* Proceedings of the 21st Annual ACM SIGIR Conference, pp. 335–336.

[3] Lundberg, S. M. & Lee, S.-I. (2017). *A unified approach to interpreting model predictions.* Advances in Neural Information Processing Systems, 30.

[4] Lewis, P. et al. (2020). *Retrieval-Augmented Generation for knowledge-intensive NLP tasks.* Advances in Neural Information Processing Systems, 33, 9459–9474.

[5] LangChain AI (2024). *LangGraph: Building stateful, multi-actor applications with LLMs.* https://github.com/langchain-ai/langgraph

[6] Chroma (2024). *Chroma: the open-source embedding database.* https://www.trychroma.com

[7] Groq Inc. (2024). *Groq LPU Inference Engine.* https://groq.com